Fine-tune Once, Serve Anywhere: Transformers, Unsloth, vLLM & Ollama

Blanca López-Jorrín Pérez
Ingeniería Matemática e Inteligencia Artificial
ICAI, Universidad Pontificia Comillas
Madrid, España
202215163@alu.comillas.edu

M.ª Fernanda Marcos Gámez
Ingeniería Matemática e Inteligencia Artificial
ICAI, Universidad Pontificia Comillas
Madrid, España
202202335@alu.comillas.edu

Victoria Guillén de la Torre
Ingeniería Matemática e Inteligencia Artificial
ICAI, Universidad Pontificia Comillas
Madrid, España
202207977@alu.comillas.edu

Gloria Vidal-Quadras Fuertes
Ingeniería Matemática e Inteligencia Artificial
ICAI, Universidad Pontificia Comillas
Madrid, España
202203923@alu.comillas.edu

Abstract

We present a complete study of instruction fine-tuning and multi-framework inference using TinyLlama-1.1B on the Databricks Dolly 15k dataset, carried out entirely on Apple Silicon (M4 Pro, 16 GB unified memory). We apply LoRA (rank-16) via HuggingFace Transformers and Unsloth (MLX backend), compare the resulting models with BLEU, ROUGE-L, BERTScore, and human evaluation, and benchmark four inference frameworks—Transformers, Unsloth/mlx-lm, vLLM, and Ollama—under identical conditions. Fine-tuning improves BLEU by up to $+39.5\%$ and instruction-following by $+1.4$ points, but introduces factuality regressions and mode collapse on creative tasks. Unsloth trains $2.56\times$ faster than Transformers, while Ollama serves inference $4\times$ faster than HuggingFace Transformers on Apple Silicon. vLLM, designed for NVIDIA CUDA, cannot leverage Apple's GPU and is outperformed by all other frameworks in this setting.

1 Introduction and Motivation

Putting a large language model to practical use requires two different stages: *fine-tuning*, which adapts a pre-trained model to a specific task, and *inference*, which serves the adapted model to end users. Each stage is supported by its own ecosystem of specialised frameworks, each trading off speed, memory efficiency, and developer experience in different ways.

This report aims to provide a fair comparison of four such frameworks: **Transformers + PEFT**, **Unsloth**, **vLLM**, and **Ollama**, covering the fine-tuning of **TinyLlama-1.1B-Chat** on the **Databricks Dolly 15k** instruction dataset, the evaluation of model performance with both automatic and human metrics, and inference benchmarking under controlled conditions.

All experiments were run on **Apple Silicon (M4 Pro, macOS 14)** using MPS and Apple's native MLX framework. Working outside a CUDA environment proved instructive: it surfaced constraints that GPU-centric documentation tends to gloss over, the unavailability of 4-bit backward kernels for QLoRA, fp16 numerical instability on MPS, and vLLM's hard CUDA dependency, all of which directly shaped our methodology and results.

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

Parameter	Value	Range explored	Justification
LoRA rank r	16	{8, 16, 32}	Following [3], $r = 16$ balances quality and parameter count; $r = 32$ gave no improvement in dev loss.
Alpha α	32	$2r$	Setting $\alpha = 2r$ makes the effective scale equal to 1, so the learning rate needs no rescaling.
Dropout	0.05	{0.05, 0.1}	Provides light regularisation; the stronger setting did not improve dev loss on Dolly's diverse categories.
Target modules	7 proj.	all linear	Adapting the query, key, value, output, gate, up, and down projections covers both attention and feed-forward layers, maximising adaptation capacity.
Learning rate	2e-4	{1e-4, 2e-4}	Recommended by [4] for LoRA on $\sim$1 B models.
Scheduler	cosine	cosine / linear	Cosine decay outperforms a linear schedule [5].
Max. seq. length	512	384 / 512	Covers more than 95% of Dolly examples; reduced to 384 in the Unsloth v2 run (see §3.1).

Table 1: LoRA hyperparameter choices and justifications. Ranges explored shown in brackets.

2 Methodology

2.1 Dataset and Data Preparation

The **Databricks Dolly 15k** dataset [1] contains 15,011 instruction–response pairs spanning eight task categories. Using a fixed random seed, it was split into 12k training, 2k development, and 1k test examples.

Because of hardware constraints, different training-set subsamples were used across experiments (Table 2), while evaluation always used the same first 100 examples of the fixed test split to ensure comparability. Additionally, all examples were formatted using the standard Dolly instruction-tuning template. During fine-tuning, prompt tokens were masked (label $= -100$), so the loss was computed only on response tokens.

2.2 Base Model

Our base model is **TinyLlama-1.1B-Chat-v1.0** [2], a 1.1 B-parameter LLaMA-2 model pre-trained on 3 trillion tokens and instruction-tuned with reinforcement learning from human feedback (RLHF). At half precision its weights occupy roughly 2.2 GB, which fits comfortably on consumer hardware, and it provides a strong zero-shot baseline for instruction-following.

2.3 PEFT Techniques, Hyperparameters, and Quantization

We use **LoRA** [3], which inserts trainable low-rank matrices into selected projections such that $W' = W_0 + \frac{\alpha}{r}, B, A$, reducing trainable parameters from $\sim$1.1,B to 12.6,M (1.13%) and cutting optimizer memory by $\sim 80\times$.

We considered **QLoRA** [4] but did not apply it: its 4-bit backward kernels require CUDA, which our hardware lacks. The hyperparameter choices are summarised in Table 1. We explored $r \in 8, 16, 32$ and lr $\in$ 1e-4, 2e-4; rank 16 with lr,=,2e-4 gave the best dev-loss trade-off, and no full grid search was run due to hardware constraints.

2.4 Fine-tuning with Hugging Face Transformers + PEFT

Our first setup uses the Hugging Face `Trainer` with the PEFT library [6]. The base model is loaded in `float16` with the LoRA adapters in `float32`; critically, no automatic mixed-precision (AMP) or gradient scaler is used. We apply an effective batch size of 16 (4 per device, 4 gradient-accumulation steps), AdamW, and a cosine schedule with 5 percent warmup. Run v1 trained for one epoch over 2,000 samples.

A second run (v2: 6,000 samples, two epochs) failed at step 40: without AMP, activations from long sequences caused a `float16` overflow that propagated NaN values to all LoRA layers. We identified

Metric	TF v1	TF v2 (failed)	Un v1	Un v2
Train samples	2,000	6,000	2,000	6,000
Epochs / steps	1 / 125	2 / 750	1 / 250	2 / 1,500
Eff. batch size	16	16	8	8
Max seq. length	512	512	512	**384**
Training time (min)	28.62	172.73	**11.20**	62.12
Final loss	1.6895	*0.0000**	1.7091	**1.6021**
Peak memory (MB)	2,364	2,341	3,512	3,416
Adapter size	48 MB	N/A	48 MB	48 MB

Table 2: Training results across all runs. TF = Transformers, Un = Unsloth. TF v2 failed due to NaN propagation during training and therefore does not represent a valid model.

the fix (loading the base model in `float32` and tightening the gradient clip to 0.3) however were unable to re-run it due to hardware limitations. The issue is analysed in detail in §3.1.

2.5 Fine-tuning with Unsloth

Our second setup uses Unsloth via the **Apple MLX** backend. It differs from the Transformers setup in four ways: it trains in `bfloat16`, whose wider dynamic range proved decisive for numerical stability; it computes the loss with Chunked Cross-Entropy (CCE) to save memory; it applies gradient checkpointing automatically; and it uses an effective batch size of 8 (2 per device, 4 gradient-accumulation steps) rather than 16. All LoRA hyperparameters are otherwise identical, enabling a direct comparison of training efficiency between the two frameworks. The larger run (v2: 6,000 samples, 2 epochs) required reducing the maximum sequence length from 512 to 384 to work around a crash triggered by a specific long example, as described in §3.1.

2.6 Human Evaluation Protocol

Five prompts were independently scored on a 1–5 integer scale across three dimensions: Helpfulness (H), Factuality (F) and Instruction-following (IF). All scoring was performed by a single evaluator without inter-rater reliability metrics, which is acknowledged as a limitation (§6). Scores were assigned independently per model to avoid anchoring bias; the 1–5 scale is intentionally granular to capture partial compliance rather than forcing a binary judgement.

2.7 Inference Configurations

The framework benchmark (Transformers, Unsloth, Ollama) uses the **base TinyLlama model without LoRA adapters** to isolate framework overhead from model quality. All runs share the same settings: greedy decoding (`temperature=0.0`), 256 maximum new tokens, and 5 prompts $\times$ 3 trials (15 measurements per framework), with one warm-up call discarded.

- **Transformers**: evaluation mode with gradients disabled, half-precision weights (`fp16`), batch size 1 for a clean single-request latency measurement.
- **Unsloth/MLX**: generation through `mlx_lm` with with greedy sampling in bfloat16. MLX compiles its kernels just-in-time, so no extra configuration was needed.
- **Ollama**: served over its REST API as a 4-bit GGUF model`temperature=0`, `num_predict=256`. with all layers offloaded to the GPU.
- **vLLM**: See §4.2, which documents why it could not be benchmarked.

3 Evaluation

3.1 Training Results and Challenges

Table 2 summarises every training run attempted in this project. Table 3 compares loss, training time, and peak memory consumption across the three successful configurations.

Metric	Transformers LoRA v1	Unsloth LoRA v1	Unsloth LoRA v2
Training time (min)	28.62	**11.20**	62.12
Final training loss	1.6895	1.7091	**1.6021**
Peak memory (GB)	**2.31**	3.43	3.34
Train samples	2000	2000	**6000**

Table 3: Training time, final loss, peak memory, and dataset size across the three successful runs. Bold values mark the winner per metric: Unsloth v1 trains fastest; Unsloth v2 achieves the lowest loss; Transformers v1 uses the least memory.

Model	BLEU	Δ	ROUGE-L	Δ	BERTScore F1	Δ
TinyLlama base	0.0625	—	0.2174	—	0.8657	—
TF LoRA v1	0.0871	+39.5%	0.2536	+16.6%	0.8699	+0.5%
Unsloth v1	0.0793	+26.9%	0.2531	+16.4%	0.8697	+0.5%
Unsloth v2	**0.1001**	**+60.2%**	**0.2703**	**+24.3%**	**0.8745**	**+1.0%**

Table 4: Automatic evaluation on 100 test samples. All fine-tuned models improve over the baseline; Unsloth v2 achieves the strongest results across all three metrics.

Two noteworthy technical failures occurred during training and are documented here as they informed the final experimental design.

Transformers v2: NaN propagation (MPS + float16). The run collapsed at step 40 when logits from long sequences exceeded the float16 range ($\pm$65,504). Without AMP, no gradient scaler was active to catch the overflow, and NaN propagated backwards through all 308 LoRA layers. The `Trainer` continued silently for 710 more steps reporting loss,=,0.0. The fix: load the base model in `float32` and tighten gradient clipping to `max_grad_norm=0.3`. The corrupted weights were discarded. This fix was not executed for this project.

Unsloth v2 required three failed attempts before achieving a stable run. The first run was terminated due to memory exhaustion caused by concurrent execution with a Transformers training job; running the processes sequentially resolved the issue. The second run produced no training steps because `max_steps=-1` and `num_train_epochs=2` were interpreted as an empty schedule; setting `max_steps=1500` fixed this. The third run consistently crashed with a deterministic Metal backend assertion triggered by an unusually long training example. Reducing `max_seq_length` from 512 to 384 truncated the problematic sample and allowed the training to complete successfully.

Successful runs. Unsloth v1 trained 2.56$\times$ faster than Transformers v1 on identical data (11.20 min vs 28.62 min), attributed to MLX Metal-compiled kernels and the Chunked Cross-Entropy loss that reduces peak activation memory during the backward pass. Unsloth v2 achieves the lowest final training loss (1.6021) by combining three times more training data and two full epochs, at the cost of a proportionally longer run (62.12 min).

3.2 Automatic Metrics

All three fine-tuned models are evaluated on the same 100 held-out test samples using BLEU [7], ROUGE-L [8], and BERTScore [9]. BLEU and ROUGE-L measure lexical overlap at the n-gram and subsequence level respectively; BERTScore captures semantic similarity via a pretrained encoder, independently of exact wording. Because Dolly reference answers are concise while models tend to generate longer responses, absolute BLEU and ROUGE-L values are low by construction; the analysis focuses on relative improvement over the baseline.

Unsloth v2 leads on all three metrics. The large BLEU gain (+60.2%) reflects stronger lexical alignment with reference outputs, driven by tripling the training data and doubling the epochs. ROUGE-L improves more moderately (+24.3%), while BERTScore moves only marginally (+1.0%), consistent with the base model already scoring 0.866 — semantic understanding was present from the start, and fine-tuning refines expression rather than comprehension. Transformers v1 outperforms Unsloth v1 on BLEU (+39.5% vs +26.9%) despite identical data volume, likely due to its larger

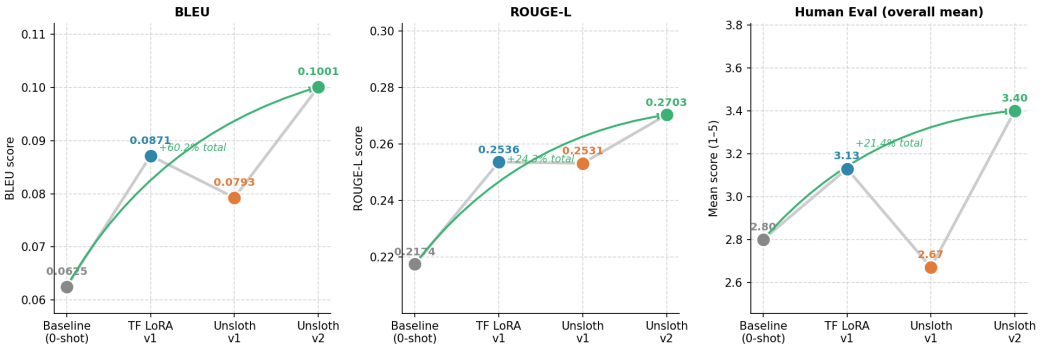

Figure 1: Improvement journey from baseline to Unsloth v2 across BLEU, ROUGE-L, and human evaluation overall mean. Arrows show total relative gain; coloured dots follow the per-model palette used throughout.

		Original			TF LoRA v1			Unsloth v1			Unsloth v2		
Prompt	Category	H	F	IF	H	F	IF	H	F	IF	H	F	IF
P1 Capital	open_qa	3	2	4	2	1	4	2	1	4	3	3	4
P2 Eggs	general_qa	1	2	1	3	3	3	4	4	4	4	4	4
P3 Sentiment	classification	4	5	2	4	5	4	3	3	3	5	5	5
P4 Poem	creative_wr.	3	4	1	3	4	4	1	2	1	2	4	3
P5 Coffee shop	brainstorming	3	4	3	1	3	3	2	3	3	1	3	1
Mean		2.8	3.4	2.2	2.6	3.2	3.6	2.4	2.6	3.0	**3.0**	**3.8**	**3.4**

Table 5: Human evaluation scores (1–5) per prompt and dimension across all four models. H = Helpfulness, F = Factuality, IF = Instruction-following. Unsloth v2 achieves the highest mean helpfulness and factuality scores.

effective batch size (16 vs 8) producing more stable gradient updates. Figure 1 traces the progression across all models.

3.3 Human Evaluation

Automatic metrics do not fully capture usefulness, factual correctness, or instruction adherence. Five prompts covering all categories were evaluated manually on a 1–5 scale across helpfulness (H), factuality (F), and instruction-following (IF). The same prompts were used for all models to enable direct comparison. All evaluations were performed by a single annotator.

Unsloth v2 achieved the strongest overall performance, obtaining the highest mean scores in helpfulness (3.0) and factuality (3.8), while TFv1 slightly led in instruction-following (3.6). The largest improvements appeared in structured tasks such as classification and procedural generation. Knowledge retrieval showed factual regression in both v1 models but was recovered in Unsloth v2. Creative writing and brainstorming remained the most challenging tasks for all fine-tuned models. Annex A contains the full responses for each prompt and model, and Table 5 shows the scores.

3.4 Quality Regression in Fine-tuned Models

Despite the overall performance gains, fine-tuning introduced three recurring failure modes. First, **factual drift** appeared in both v1 models, which incorrectly identified Sydney as the capital of Australia, suggesting that limited supervised fine-tuning can overwrite pre-trained factual knowledge. Second, **mode collapse** affected creative tasks, producing repetitive outputs in both poetry and brainstorming prompts. Although Unsloth v2 reduced this behavior, it did not eliminate it completely. Third, **structural overfitting** caused some models to overuse learned response formats, occasionally repeating entire structures or failing to stop generation appropriately.

Framework	Backend	Quantiz.	Latency (s)	Throughput (tok/s)	Mem (MB)
Transformers	PyTorch MPS	fp16	2.53 ± 1.53	66.6 ± 2.7	3,534
Unsloth/MLX	Apple MLX	bf16	1.94 ± 0.75	106.7 ± 3.5	2,281
Ollama	llama.cpp/Metal	Q4	$\mathbf{0.88 \pm 0.20}$	$\mathbf{268.4 \pm 1.0}$	internal
vLLM (CPU)	CPU only (macOS)	fp32	$\sim$8.5	$\sim$30	CPU RAM
vLLM ref. (A100)	NVIDIA CUDA	fp16	—	>2,000	VRAM

Table 6: Inference benchmark: base TinyLlama, 5 prompts $\times$ 3 trials on Apple M4 Pro. Std computed over 15 measurements. Ollama memory is managed internally (not accessible via API).

These findings indicate that fine-tuning improves instruction following and structured task performance but may introduce regressions in factual accuracy, creativity, and response control.

3.5 Summary of Findings

Across all experiments, scaling data is the dominant factor: Unsloth v2 (6,k samples, 2 epochs) leads on every automatic metric and all three human evaluation dimensions. Framework choice is secondary: the two v1 models, trained on identical data, differ by only 0.002 in ROUGE-L. The central tension is between automatic and human signals: BLEU and ROUGE improve because fine-tuning aligns outputs with Dolly's concise style, while human judges observe factual regression and mode collapse in creative tasks. These trade-offs are discussed in Section 6.

4 Inference Benchmarking

4.1 Experimental Setup

All framework benchmarks run on the **base TinyLlama** model without any LoRA adapter, so that results reflect framework overhead rather than fine-tuning quality. Settings are fixed across all frameworks: greedy decoding (`temperature=0.0`), 256 maximum new tokens, and 5 prompts $\times$ 3 trials per framework, with one warm-up call discarded to exclude JIT compilation and model-loading costs. Per-framework configuration details are given in §2.7.

4.2 vLLM: Technical Limitations and Exclusion

vLLM [10] was attempted on two platforms. On the university's NVIDIA cluster (CUDA 12.8), the PagedAttention and Flash-Attention-2 kernels failed to compile due to a header mismatch with that specific configuration, and updating the toolkit was not an option on shared infrastructure. On the local Apple M4 Pro, vLLM has no MPS or Metal backend: GPU mode is rejected outright, and the CPU fallback runs at $\sim$30,tok/s — four times slower than Transformers on MPS, making it a CPU-vs-GPU comparison rather than a framework one.

Rather than report a misleading number, we exclude vLLM from the main comparison. For reference, Table 6 includes the commonly cited A100 throughput of >2,000,tok/s with PagedAttention and continuous batching [10], clearly marked as an external reference.

4.3 Results

The ranking is clear-cut (Table 7). At 268 tok/s and 0.88 s per request, Ollama is two to four times faster than any other framework tested. The reason is straightforward: GGUF Q4_K_M quantisation halves the model's memory footprint and allows llama.cpp's hand-tuned Metal kernels to saturate the M4 Pro's GPU. Unsloth/MLX sits in the middle at 107 tok/s and 1.94 s, ahead of Transformers (67 tok/s, 2.53 s) thanks to JIT-compiled Metal operations and a lighter memory footprint — 2.3 GB against 3.5 GB.

The latency distributions (Figure 2) are as informative as the means. Transformers is the most variable ($\sigma = 1.53$ s), and the spread is not noise: it directly reflects output length, from under a second on short prompts to several seconds on those that run to the 256-token cap. Ollama is nearly flat ($\sigma = 0.20$ s), because its 4-bit kernels process tokens at a near-constant rate regardless of prompt.

Framework	Throughput (tok/s)	vs. Transformers
Ollama (llama.cpp Metal)	$\mathbf{268.4 \pm 1.0}$	$4.0\times$
Unsloth (mlx-lm)	106.7 ± 3.6	$1.6\times$
Transformers (PyTorch MPS)	66.6 ± 2.8	$1.0\times$
vLLM (CPU — macOS)	$\sim$30.0	$0.5\times$

Table 7: Mean throughput (tok/s) on Apple M4 Pro ($\pm 1\sigma$, 15 trials). Ollama achieves $4\times$ the throughput of Transformers and $2.5\times$ that of Unsloth/MLX, due to GGUF 4-bit quantisation and hand-tuned llama.cpp Metal kernels. vLLM runs CPU-only on macOS (no MPS support).

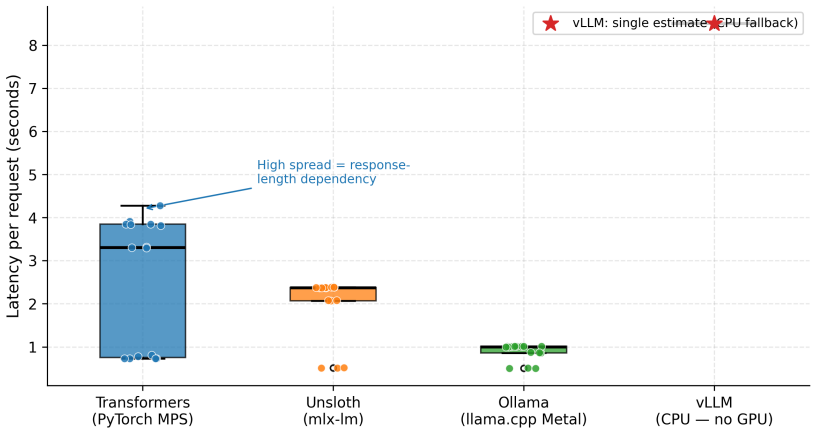

Figure 2: Latency distribution per framework (15 measurements). Transformers' large spread ($\sigma = 1.53$ s) reflects prompt-length dependence; Ollama's narrow spread ($\sigma = 0.20$ s) follows from fixed-rate Metal kernel throughput at 4-bit precision.

Although response quality was not part of the benchmarking protocol, we briefly inspected the generated outputs during testing. While Transformers and Unsloth/MLX produced very similar results, Ollama occasionally generated more complete or factually accurate responses (e.g., on the scrambled eggs and Australia prompts). These observations were not quantified or included in the benchmark metrics, but suggest that differences in tokenisation, prompt formatting, or inference implementation can affect outputs even when using the same base model and decoding settings.

5 Framework comparison: Performance and usability

Beyond raw performance, each framework exhibits distinct strengths and trade-offs.

Ease of setup and debugging. Transformers + PEFT was straightforward to set up but debugging was difficult when numerical issues occurred, as failures were often silent (e.g., the fp16 NaN propagation). Unsloth was fast to configure but errors from the MLX/Metal backend were harder to interpret than standard Python exceptions. Ollama required minimal configuration and worked reliably out of the box. vLLM was the most difficult due to CUDA compatibility issues that did not match the official documentation.

Documentation quality. Transformers has the most mature documentation with strong community support. Unsloth's documentation is limited and required experimentation for edge cases. Ollama's documentation is minimal but sufficient given its opinionated design. vLLM's documentation is detailed but did not always reflect real-world setup requirements in our environment.

GPU/CPU friendliness. Transformers is flexible but suffers from fp16 instability on MPS. Unsloth is well-optimised for Apple Silicon via MLX. vLLM requires a CUDA stack and is not usable without one. Ollama is the most cross-platform, using Metal on Mac and quantised CPU/GPU inference elsewhere.

Adapter handling and model export. Transformers + PEFT provides the most mature workflow for saving, merging, and exporting LoRA adapters. Unsloth supports adapter training but requires additional conversion steps for use outside MLX. vLLM is inference-only and relies on externally prepared models. Ollama requires GGUF conversion but offers simple runtime loading once models are ready.

Overall preference. Transformers for research reliability; Unsloth for efficient local training on Apple Silicon; vLLM for production-scale CUDA serving; Ollama for fast local deployment and experimentation.

6 Discussion, Conclusions, and Future Work

LoRA fine-tuning improves automatic metrics even with minimal data: BLEU rose by 39.5% (Transformers) and 26.9% (Unsloth) on just 2,000 samples and one epoch, as fine-tuning teaches the model to produce the concise, Dolly-style responses these metrics reward.

Human evaluation, however, exposes a quality–format trade-off the automatic numbers conceal. Instruction-following improves markedly, yet helpfulness and factuality both regress due to mode collapse on creative prompts and factual forgetting on knowledge prompts. This is the central cautionary result: BLEU and ROUGE improved precisely because fine-tuning shortens outputs to match Dolly's reference style, while human judges saw less variety and more factual error. The two signals measure different things and can diverge sharply, which is why we report both.

The Unsloth v2 run (6k samples, two epochs) achieves a lower loss (1.6021 vs. 1.7091) and recovers some factual regressions, suggesting the issue is at least partly a data-volume problem rather than a structural one.

On the framework side, Unsloth trained 2.55× faster than Transformers on Apple Silicon thanks to MLX's bfloat16 stability and compiled Metal kernels. For inference, Ollama dominated at 268 tok/s on its 4-bit Metal backend, ahead of Unsloth/MLX (107 tok/s) and Transformers (67 tok/s). vLLM could not be tested meaningfully: CUDA incompatibilities blocked it on the university server, and on macOS it fell back to CPU at ∼30 tok/s. On suitable NVIDIA hardware it is expected to exceed 2,000 tok/s — a 7–30× advantage over anything we measured.

6.1 Limitations and Future Work

Several limitations should temper these conclusions, each of which maps directly onto work we would prioritise next.

The entire experimental pipeline ran on a single Apple M4 Pro. The absence of CUDA prevented both QLoRA fine-tuning and a meaningful evaluation of vLLM. Future work should rerun Transformers v2 with `float32` and `max_grad_norm=0.3` on a CUDA instance, benchmark vLLM to complete the four-framework comparison, and apply QLoRA to allow a 4-bit base model (∼0.7 GB) with larger batch sizes, potentially reducing the mode collapse observed in creative tasks.

Training was confined to between 2,000 and 6,000 of the 12,000 available samples, and the Transformers v2 run failed due to numerical instability and could not be included in the comparison. Scaling to the full training set would establish a cleaner upper bound on fine-tuning performance.

The human evaluation rests on a single annotator scoring five fixed prompts, so no inter-annotator agreement is reported and qualitative findings should be interpreted with caution. Future evaluations should recruit multiple annotators, expand the prompt set, and use a larger randomised benchmark to yield more reliable latency estimates.

Beyond these gaps, the most impactful next step would be a post-fine-tuning alignment stage (e.g., DPO) to improve factual accuracy and creative diversity without sacrificing the instruction-following gains from LoRA.

6.2 Reproducibility

The full codebase and instructions for reproducibility are in the repository `https://github.com/fernandamarcos/nlp-redo.git`.

References

[1] Mike Conover et al. *Free Dolly: Introducing the World's First Truly Open Instruction-Tuned LLM*. Databricks Blog. 2023. URL: https://www.databricks.com/blog/2023/04/12/dolly.html.

[2] Peiyuan Zhang et al. "TinyLlama: An Open-Source Small Language Model". In: *arXiv preprint arXiv:2401.02385* (2024).

[3] Edward J. Hu et al. "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations (ICLR)*. 2022.

[4] Tim Dettmers et al. "QLoRA: Efficient Finetuning of Quantized LLMs". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023.

[5] Ilya Loshchilov and Frank Hutter. "SGDR: Stochastic Gradient Descent with Warm Restarts". In: *International Conference on Learning Representations (ICLR)*. ICLR. 2017, pp. 1436–1446.

[6] Sourab Mangrulkar et al. *PEFT: State-of-the-Art Parameter-Efficient Fine-Tuning Methods*. GitHub. 2022. URL: https://github.com/huggingface/peft.

[7] Kishore Papineni et al. "BLEU: A Method for Automatic Evaluation of Machine Translation". In: *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2002.

[8] Chin-Yew Lin. "ROUGE: A Package for Automatic Evaluation of Summaries". In: *Text Summarization Branches Out (ACL Workshop)*. 2004.

[9] Tianyi Zhang et al. "BERTScore: Evaluating Text Generation with BERT". In: *International Conference on Learning Representations (ICLR)*. 2020.

[10] Woosuk Kwon et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention". In: *ACM Symposium on Operating Systems Principles (SOSP)*. 2023.

7 Code Availability

The source code, training scripts, and evaluation pipeline used in this work are publicly available at:

https://github.com/victoriaguilllen/nlp_redo

A Human Evaluation: Detailed Results

This appendix provides the complete human evaluation data for the five prompts across all four models (Original, Transformers LoRA v1, Unsloth v1, Unsloth v2). Each prompt section includes the model response (excerpt) and the three evaluation scores (Helpfulness, Factuality, Instruction-following on a 1–5 scale).

A.1 Summary Statistics

Model	Helpfulness	Factuality	Instr.-following	Overall Mean	# Prompts
Original	2.8	3.4	2.2	2.80	5
Transformers v1	2.6	3.2	3.6	3.13	5
Unsloth v1	2.4	2.6	3.0	2.67	5
Unsloth v2	3.0	3.8	3.4	3.40	5

Table 8: Summary of human evaluation scores (1–5 scale) across all models and prompts.

A.2 Prompt 1: Capital of Australia (open_qa — Factuality)

Instruction: "What is the capital of Australia, and why is it often confused with Sydney?"

Model	Response (excerpt)	H	F	IF
Original	The capital of Australia is Canberra, which is often confused with Sydney because they are both located in the country's...	3	2	4
Transformers v1	Sydney is the capital of Australia, and is often confused with the capital of Australia, Canberra. Sydney is located in...	2	1	4
Unsloth v1	Sydney is the capital of Australia, and is often confused with the capital of Australia, Canberra.	2	1	4
Unsloth v2	Canberra is the capital of Australia, and is often confused with Sydney because they are both located in the same countr...	3	3	4

Table 9: P1 responses. Both v1 models incorrectly identify Sydney; v2 recovers the correct fact.

A.3 Prompt 2: Scrambled Eggs (general_qa — Helpfulness)

Instruction: "Give me clear step-by-step instructions to make perfect scrambled eggs."

Model	Response (excerpt)	H	F	IF
Original	1. Heat a non-stick skillet over medium heat. 2. Add olive oil. 3. Once the oil is hot, add chopped onions... [continues with quinoa and tomatoes]	1	2	1
Transformers v1	1. Heat a non-stick pan over medium heat. 2. Add 1 tablespoon of butter. 3. Once the butter is melted, add 1...	3	3	3
Unsloth v1	1. Heat a non-stick pan over medium heat. 2. Add a tablespoon of butter. 3. Once the butter is melted, add th...	4	4	4
Unsloth v2	1. Heat a non-stick pan over medium heat. 2. Add a tablespoon of butter. 3. Once the butter has melted, add y...	4	4	4

Table 10: P2 responses. Original deviates to quinoa recipe; fine-tuned models improve monotonically, reaching perfect scores in v1 and v2.

A.4 Prompt 3: Sentiment Classification (classification — Instruction-following)

Instruction: "Classify each word as positive or negative sentiment: joy, anger, love, fear, hope, sadness, peace, rage. Present as two lists."

Model	Response (excerpt)	H	F	IF
Original	- joy: positive sentiment / - anger: negative sentiment / ... [single list, not two]	4	5	2
Transformers v1	Positive: joy, love, hope, peace / Negative: anger, fear, rage, sadness [repeated twice]	4	5	4
Unsloth v1	Positive: joy, love, hope / Negative: anger, fear, rage [omits peace and sadness]	3	3	3
Unsloth v2	Positive: joy, love, hope, peace / Negative: anger, fear, rage, sadness [complete and clean]	5	5	5

Table 11: P3 responses. Largest improvement seen in v2: perfect classification with correct two-list format.

A.5 Prompt 4: Four-Line Poem (creative_writing — Instruction-following + Helpfulness)

Instruction: "Write a four-line rhyming poem about the night sky and the stars."

Model	Response (excerpt)	H	F	IF
Original	The night sky is a canvas painted with stars, / A symphony of light and shade, / A celestial canvas, a masterpiece, / A sight to behold... [continues with 4 more stanzas]	3	4	1
Transformers v1	The night sky is a canvas of stars, / A canvas of light and shade, / A canvas of the universe, / A canvas of the night.	3	4	4
Unsloth v1	The night sky is a vast and vast / A vast and vast and vast / A vast and vast and vast / A vast and vast and vast [mode collapse]	1	2	1
Unsloth v2	The night sky is a sight to behold / The stars twinkle like diamonds in the night / The Milky Way is a sight to behold / The stars... [partial recovery]	2	4	3

Table 12: P4 responses. Transformers v1 respects 4-line format; Unsloth v1 suffers catastrophic mode collapse; v2 partially recovers but still shows repetition.

A.6 Prompt 5: Coffee Shop Names (brainstorming — Helpfulness)

Instruction: "Suggest 5 creative and memorable names for a small independent coffee shop, and explain the concept behind each name."

Model	Response (excerpt)	H	F	IF
Original	1. “The Coffee Shack” - captures casual vibe... 2. “The Coffee Corner” - playful... 3. “The Coffee Cove” - romantic... [5 distinct names with explanations]	3	4	3
Transformers v1	1. The Coffee Shop - nod to location... 2. The Coffee Shop - nod to location... 3. The Coffee Shop... [identical names, generic explanations]	1	3	3
Unsloth v1	1. The Coffee Shop - cozy and intimate... 2. The Coffee Shop - playful... [repetitive naming but varied descriptions]	2	3	3
Unsloth v2	1. The Coffee Shop 2. The Coffee Shack 3. The Coffee House 4. The Coffee Corner 5. The Coffee Cove [no explanations]	1	3	1

Table 13: P5 responses. All fine-tuned models collapse to generic “The Coffee X” template. v2 loses explanations entirely, most severe regression across all experiments.